CONNECT

AI-Powered Engineering Collaboration Platform

Full System Design & Build Document

Version 1.0 | MVP-First Architecture | Production-Ready

---

0. The Problem Being Solved

CONNECT exists to solve one specific, painful problem for engineering students:

> ***"I have a final year project idea but spent 3 weeks looking for a teammate who knows React.***
> ***I ended up doing it alone and it failed."***

That is what CONNECT solves. Not 'fragmented collaboration' in the abstract. This specific thing: a student cannot find the right teammate, with the right skill, who is available, for their specific project — and they have no tool built for that.

What existing tools miss:

- WhatsApp: no skill indexing, no project structure, no AI
- LinkedIn: not for students, not real-time, no project context
- Notion: no people-finding, no AI matching
- GitHub: no social layer, no team formation

CONNECT combines all of this into one platform, built for engineering students and professionals.

---

1. System Overview

1.1 What CONNECT Is

CONNECT is a monolithic-first, AI-assisted, real-time collaboration platform. It is not a graph database system — it is a relational system with AI layered on top. It is not a microservices architecture at launch — that comes only after proven traction.

Platform Type	Monolithic SaaS Web App (MVP), scalable to microservices post-traction
Primary Users	Engineering students, teachers, professionals
Core Value	Find the right teammate + collaborate in one place + AI guidance
AI Role	Team matching, project advisory, semantic search, code help
Launch Target	4–6 weeks to working MVP with 50 real users

1.2 Architecture (Correct, Non-Contradictory)

> *IMPORTANT: Do NOT split into microservices on day one. Build as a single Express app. The AI service lives as a folder inside the backend, not a separate deployed service. Extract only after you have real scaling pain.*

```
BROWSER
   |
   v
NEXT.JS FRONTEND (Vercel)
   |
   v (HTTPS)
EXPRESS BACKEND — single app, modular routes (Render / Railway)
   |
   +——> User Service (auth, profiles, skills)
   |
   +——> Project Service (projects, teams, tasks)
   |
   +——> Chat Service (WebSockets + Redis Pub/Sub)
   |
   +——> AI Service (OpenAI API + embeddings) <-- folder, not separate deploy
   |
   +——> Search Service (PostgreSQL full-text, later pgvector)
   |
   v
POSTGRESQL (Supabase / Neon — free tier)
   +
REDIS (Upstash — free tier for WebSockets)
   +
AWS S3 (file storage)
```

2. Database Schema (Fixed & Complete)

2.1 ER Relationships

These are the actual foreign key relationships between tables:

```
users          ──< profiles         (1:1, user_id FK)
users          ──< user_skills       (1:many, user_id FK)
skills         ──< user_skills       (1:many, skill_id FK)
users          ──< projects          (1:many, created_by FK)
projects       ──< teams             (1:1, project_id FK)
teams          ──< team_members      (1:many, team_id FK)
users          ──< team_members      (1:many, user_id FK)
projects       ──< tasks             (1:many, project_id FK)
users          ──< tasks             (1:many, assigned_to FK)
projects       ──< chat_rooms        (1:1, project_id FK)
chat_rooms     ──< messages          (1:many, room_id FK)  <-- FIXED
users          ──< messages          (1:many, sender_id FK)
users          ──< notifications     (1:many, user_id FK)
users          ──< resources         (1:many, uploaded_by FK)
```

2.2 Full SQL Schema

users

```sql
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       VARCHAR(255) UNIQUE NOT NULL,
  password    VARCHAR(255),  -- nullable for OAuth users
  role        VARCHAR(20) NOT NULL DEFAULT 'student'
              CHECK (role IN ('student','teacher','professional','admin')),
  provider    VARCHAR(20) DEFAULT 'local'
              CHECK (provider IN ('local','google')),
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

profiles

```sql
CREATE TABLE profiles (
  user_id      UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  name         VARCHAR(100) NOT NULL,
  institution  VARCHAR(150),
  department   VARCHAR(100),
  year         INTEGER CHECK (year BETWEEN 1 AND 6),
  bio          TEXT,
  avatar_url   TEXT,
  github_url   TEXT,
  linkedin_url TEXT,
```

```
  skill_vector VECTOR(1536),  -- pgvector embedding, added later
  updated_at   TIMESTAMPTZ DEFAULT NOW()
);
```

skills + user_skills

```
CREATE TABLE skills (
  id         SERIAL PRIMARY KEY,
  name       VARCHAR(100) UNIQUE NOT NULL,
  category   VARCHAR(50)  -- 'frontend','backend','ml','design', etc.
);

CREATE TABLE user_skills (
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
  skill_id    INTEGER REFERENCES skills(id),
  level       VARCHAR(20) CHECK (level IN ('beginner','intermediate','advanced')),
  PRIMARY KEY (user_id, skill_id)
);
```

projects + teams + team_members

```
CREATE TABLE projects (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title         VARCHAR(200) NOT NULL,
  description   TEXT,
  required_skills TEXT[],  -- array of skill names for matching
  status        VARCHAR(20) DEFAULT 'open'
                CHECK (status IN ('open','in_progress','completed')),
  created_by    UUID REFERENCES users(id),
  created_at    TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE teams (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  UUID UNIQUE REFERENCES projects(id) ON DELETE CASCADE,
  max_size    INTEGER DEFAULT 5
);

CREATE TABLE team_members (
  team_id   UUID REFERENCES teams(id) ON DELETE CASCADE,
  user_id   UUID REFERENCES users(id) ON DELETE CASCADE,
  role      VARCHAR(50) DEFAULT 'member',
  joined_at TIMESTAMPTZ DEFAULT NOW(),
  PRIMARY KEY (team_id, user_id)
);
```

tasks

```
CREATE TABLE tasks (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  UUID REFERENCES projects(id) ON DELETE CASCADE,
  assigned_to UUID REFERENCES users(id),
  title       VARCHAR(200) NOT NULL,
  description TEXT,
  status      VARCHAR(20) DEFAULT 'todo'
              CHECK (status IN ('todo','in_progress','done')),
  deadline    DATE,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

chat_rooms + messages (FIXED — routing gap resolved)

> *Previous schema had no room_id in messages. A message had no way to know which conversation it belonged to. This schema fixes that with explicit chat_rooms table.*

```
CREATE TABLE chat_rooms (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  UUID UNIQUE REFERENCES projects(id) ON DELETE CASCADE,
  type        VARCHAR(20) DEFAULT 'team'
              CHECK (type IN ('team','anonymous')),
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE messages (
  id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id    UUID NOT NULL REFERENCES chat_rooms(id) ON DELETE CASCADE,
  sender_id  UUID REFERENCES users(id),  -- NULL for anonymous
  content    TEXT NOT NULL,
  is_anon    BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

resources

```
CREATE TABLE resources (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  uploaded_by  UUID REFERENCES users(id),
  project_id   UUID REFERENCES projects(id),  -- nullable, can be global
  title        VARCHAR(200) NOT NULL,
  type         VARCHAR(20) CHECK (type IN ('note','file','tutorial','link')),
  file_url     TEXT,
  created_at   TIMESTAMPTZ DEFAULT NOW()
);
```

notifications

```
CREATE TABLE notifications (
  id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id    UUID REFERENCES users(id) ON DELETE CASCADE,
  type       VARCHAR(50),  -- 'team_invite','project_update','message'
  payload    JSONB,        -- flexible data for any notification type
  is_read    BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

---

3. AI System Design (Precise Definitions)

> *Previous documents called AI the 'core differentiator' but gave it no concrete implementation. This section defines exactly how each AI feature works — inputs, outputs, algorithm. Build to this spec, not to vague descriptions.*

3.1 Skill Matching Algorithm (Defined)

The matching algorithm computes a compatibility score between a user and a project. Here is the exact logic:

```
INPUTS:
  - user_skills: [{skill_name, level}]  e.g. [{React, advanced}, {Node,
intermediate}]
  - project.required_skills: [skill_name]  e.g. ['React', 'Python', 'Node']

ALGORITHM:
  1. skill_overlap = count of user skills that appear in required_skills
  2. coverage_ratio = skill_overlap / length(required_skills)
     e.g. user has React + Node = 2 of 3 required => 0.66

  3. level_bonus = sum of level weights for matched skills
     beginner = 0.5, intermediate = 1.0, advanced = 1.5
     Normalized: level_bonus / (matched_count * 1.5)

  4. match_score = (coverage_ratio * 0.6) + (normalized_level_bonus * 0.4)
     Range: 0.0 to 1.0

OUTPUT:
  [{ user_id, name, avatar, match_score, matched_skills, missing_skills }]
  Sorted by match_score descending. Return top 10.

EXAMPLE:
  User: React(advanced), Node(intermediate)
  Project needs: React, Python, Node
  coverage_ratio = 2/3 = 0.66
  level_bonus = (1.5 + 1.0) / (2 * 1.5) = 2.5/3 = 0.83
```

```
  match_score = (0.66 * 0.6) + (0.83 * 0.4) = 0.396 + 0.332 = 0.73
```

Phase 2 improvement: Replace coverage_ratio with cosine similarity of OpenAI embeddings of skill descriptions. This handles 'React' vs 'ReactJS' vs 'frontend development' as semantically similar. Add this only after MVP is live.

3.2 AI Project Advisor

```
ENDPOINT: POST /api/ai/advisor

INPUT: { idea: string }
  e.g. 'Build a plant disease detection app using phone camera'

SYSTEM PROMPT:
  'You are a senior engineering mentor. Given a project idea,
   return a structured JSON with:
   - summary: one sentence
   - recommended_stack: [tech names]
   - phases: [{phase_name, tasks[], duration_weeks}]
   - required_skills: [skill names]
   - risks: [string]
   Respond ONLY with valid JSON. No markdown, no explanation.'

OUTPUT: Parsed JSON rendered as a visual roadmap in frontend

VALIDATION: If JSON parse fails, return generic error. Log the raw
response for debugging. Never show raw LLM output to user.
```

3.3 AI Solution Generator

```
ENDPOINT: POST /api/ai/solve

INPUT: { problem: string, language?: string }

SYSTEM PROMPT:
  'You are a senior software engineer. Given a problem description,
   provide: explanation (plain English), code solution (clean, commented),
   and 2 alternative approaches. Format as JSON:
   { explanation, code, language, alternatives: [{name, description}] }'

RATE LIMIT: 10 requests per user per hour (Redis counter)
COST CONTROL: Use gpt-3.5-turbo for this endpoint, gpt-4 only for advisor
```

3.4 Semantic Search (Phase 2 — Not MVP)

> *Do NOT build vector search for MVP. Use PostgreSQL full-text search (tsvector). Add pgvector + embeddings only after you have 200+ resources and users complaining search is bad.*

```
MVP SEARCH: PostgreSQL full-text
  SELECT * FROM projects
  WHERE to_tsvector('english', title || ' ' || description)
  @@ plainto_tsquery('english', $1);

PHASE 2 SEARCH: pgvector
  1. On resource upload: embed content via OpenAI
     embedding = await openai.embeddings.create({ input: text })
  2. Store in resources.embedding VECTOR(1536)
  3. Query: SELECT * FROM resources
     ORDER BY embedding <=> query_embedding LIMIT 10;
```

---

4. API Endpoints (Complete Reference)

4.1 Auth Routes

Method	Endpoint	Description
POST	/api/auth/register	Create account (email + password)
POST	/api/auth/login	Login, return JWT + refresh token
GET	/api/auth/google	Google OAuth redirect
GET	/api/auth/google/callback	Google OAuth callback, issue JWT
POST	/api/auth/refresh	Refresh access token
POST	/api/auth/logout	Invalidate refresh token

4.2 User & Profile Routes

Method	Endpoint	Description
GET	/api/users/:id	Get user profile (public)
PUT	/api/users/profile	Update own profile (auth)
POST	/api/users/skills	Add skills to profile (auth)
DELETE	/api/users/skills/:id	Remove skill (auth)
GET	/api/users/search?q=	Search users by name/skill

4.3 Project Routes

Method	Endpoint	Description
POST	/api/projects	Create project (auth)
GET	/api/projects	List all open projects
GET	/api/projects/:id	Get project detail
PUT	/api/projects/:id	Update project (owner)
DELETE	/api/projects/:id	Delete project (owner)
POST	/api/projects/:id/join	Request to join project
POST	/api/projects/:id/invite	Invite user to project
GET	/api/projects/:id/match	Get AI-matched users for project
POST	/api/projects/:id/tasks	Create task in project
PUT	/api/projects/:id/tasks/:tid	Update task status

4.4 Chat Routes

Method	Endpoint	Description
GET	/api/chat/rooms/:roomId/messages	Load last 50 messages
WS	ws://server/chat/:roomId	WebSocket connection for real-time chat
POST	/api/chat/anonymous	Post anonymous message to global room

4.5 AI Routes

Method	Endpoint	Description
POST	/api/ai/advisor	Generate project roadmap from idea
POST	/api/ai/solve	Generate code/explanation for problem
GET	/api/ai/match/:projectId	Return ranked user matches for project

4.6 Resource & Search Routes

Method	Endpoint	Description
POST	/api/resources	Upload file/note (multipart, S3)
GET	/api/resources?projectId=	List resources by project
GET	/api/search?q=&type=	Search users + projects (full-text)

5. Repository Structure

> *No separate ai-service folder in MVP. AI logic lives inside backend/src/services/ai/. Extract to a separate service only after you hit OpenAI timeout issues at scale.*

```
CONNECT/
├── frontend/                    # Next.js
│   ├── src/
│   │   ├── app/                 # App router pages
│   │   │   ├── page.jsx         # Landing page
│   │   │   ├── dashboard/
│   │   │   ├── profile/[id]/
│   │   │   ├── projects/
│   │   │   ├── chat/
│   │   │   └── ai/
│   │   ├── components/
│   │   │   ├── ui/              # Buttons, inputs, cards
│   │   │   ├── layout/          # Sidebar, navbar, shell
│   │   │   ├── project/         # ProjectCard, TaskBoard
│   │   │   ├── chat/            # ChatWindow, MessageBubble
│   │   │   └── ai/              # AdvisorPanel, MatchResults
│   │   ├── hooks/               # useAuth, useSocket, useProjects
│   │   ├── lib/                 # axios instance, socket.io client
│   │   └── context/             # AuthContext
│   ├── .env.local
│   └── package.json
│
├── backend/                     # Express (single app)
│   ├── src/
│   │   ├── routes/
│   │   │   ├── auth.routes.js
│   │   │   ├── user.routes.js
│   │   │   ├── project.routes.js
│   │   │   ├── chat.routes.js
│   │   │   ├── ai.routes.js
│   │   │   ├── resource.routes.js
│   │   │   └── search.routes.js
│   │   ├── controllers/         # Route handlers
│   │   ├── services/
│   │   │   ├── auth.service.js
│   │   │   ├── user.service.js
│   │   │   ├── project.service.js
│   │   │   ├── chat.service.js
│   │   │   ├── ai/
│   │   │   │   ├── advisor.js    # Project roadmap generation
│   │   │   │   ├── matcher.js    # Skill matching algorithm
│   │   │   │   └── solver.js     # Code/solution generation
│   │   │   ├── storage.service.js  # S3 upload
│   │   │   └── search.service.js
│   │   ├── middleware/
```

```
│   │   │   ├── auth.middleware.js  # JWT verify
│   │   │   ├── rbac.middleware.js  # Role checks
│   │   │   ├── rateLimit.js
│   │   │   └── validate.js        # Input sanitization
│   │   ├── models/               # DB query functions (no ORM)
│   │   ├── websocket/
│   │   │   └── chat.handler.js   # Socket.io handlers
│   │   ├── config/
│   │   │   ├── db.js             # pg pool
│   │   │   └── redis.js          # ioredis client
│   │   └── utils/
│   ├── server.js
│   └── package.json
│
├── database/
│   ├── schema.sql
│   └── seed.sql
│
├── docker/
│   ├── Dockerfile.backend
│   └── docker-compose.yml
│
├── .env.example
└── .github/workflows/deploy.yml
```

---

6. Authentication Architecture

> *Do not build auth from scratch. Use a proven library stack: bcrypt for hashing, jsonwebtoken for JWT, passport.js for Google OAuth. Building your own crypto is how you get breached.*

```
LIBRARIES:
  bcryptjs         -- password hashing (10 salt rounds)
  jsonwebtoken     -- JWT sign + verify
  passport         -- auth middleware
  passport-google-oauth20  -- Google OAuth strategy
  express-session  -- session for OAuth flow only

TOKEN STRATEGY:
  access_token:   JWT, expires 15 minutes, sent in Authorization header
  refresh_token:  opaque string, stored in DB, expires 7 days,
                  sent as httpOnly cookie (NOT in localStorage)

FLOW (Email):
  Register: hash password -> insert user -> insert profile -> return tokens
  Login:    find user -> bcrypt.compare -> if match, issue tokens

FLOW (Google):
  Redirect to Google -> callback -> find or create user -> issue tokens
```

```
  provider field = 'google', password = null

RBAC MIDDLEWARE:
  const requireRole = (roles) => (req, res, next) => {
    if (!roles.includes(req.user.role)) return res.status(403).json({error:
'Forbidden'});
    next();
  }

USAGE:
  router.post('/projects', authenticate, requireRole(['student','professional']),
...)
```

---

7. Real-Time Chat System

```
LIBRARY: socket.io (server) + socket.io-client (frontend)
CACHE: Redis (Upstash free tier) for pub/sub across instances

SERVER SETUP:
  const io = new Server(httpServer, { cors: { origin: process.env.FRONTEND_URL }
});
  const pubClient = new Redis(process.env.REDIS_URL);
  const subClient = pubClient.duplicate();
  io.adapter(createAdapter(pubClient, subClient));

SOCKET EVENTS:
  CLIENT → SERVER:
    join_room    { roomId }           -- join a chat room
    send_message { roomId, content, isAnon }
    typing       { roomId }

  SERVER → CLIENT:
    receive_message  { id, sender, content, createdAt, isAnon }
    user_typing      { userName }
    room_joined      { roomId, lastMessages[] }

PERSISTENCE:
  On send_message: insert into messages table, then emit to room.
  Messages are stored permanently. WebSocket is for delivery, not storage.

ANONYMOUS CHAT:
  isAnon = true -> sender_id = null in DB, display as 'Anonymous'
  No IP logging. No linking back to user in MVP.
  Moderation: flag system only (Phase 2)
```

---

8. Environment Configuration

8.1 Backend .env

```
# Server
PORT=5000
NODE_ENV=development
FRONTEND_URL=http://localhost:3000

# Database
DATABASE_URL=postgresql://user:pass@localhost:5432/connect_db

# Auth
JWT_SECRET=your-256-bit-secret-min-32-chars
JWT_EXPIRES_IN=15m
REFRESH_TOKEN_EXPIRES_IN=7d

# Google OAuth
GOOGLE_CLIENT_ID=your-google-client-id
GOOGLE_CLIENT_SECRET=your-google-client-secret
GOOGLE_CALLBACK_URL=http://localhost:5000/api/auth/google/callback

# Redis
REDIS_URL=redis://localhost:6379

# OpenAI
OPENAI_API_KEY=sk-...
OPENAI_MODEL_ADVISOR=gpt-4
OPENAI_MODEL_SOLVER=gpt-3.5-turbo

# AWS S3
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
AWS_REGION=ap-south-1
S3_BUCKET_NAME=connect-resources

# Rate Limiting
AI_RATE_LIMIT_PER_HOUR=10
```

8.2 Frontend .env.local

```
NEXT_PUBLIC_API_URL=http://localhost:5000/api
NEXT_PUBLIC_SOCKET_URL=http://localhost:5000
NEXT_PUBLIC_APP_NAME=CONNECT
```

---

9. Deployment (MVP — Free / Low Cost)

Frontend	Vercel — free tier, auto-deploys from GitHub main

Backend	Render.com — free tier, Docker or Node build
Database	Neon.tech or Supabase — free PostgreSQL, handles migrations
Redis	Upstash — free tier, 10k commands/day, sufficient for MVP
File Storage	AWS S3 — pay per use, negligible for MVP scale
AI	OpenAI API — pay per token, use gpt-3.5-turbo to minimize cost

9.1 Docker Compose (Local Dev)

```yaml
version: '3.8'
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: connect_db
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: password
    ports:
      - '5432:5432'
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./database/schema.sql:/docker-entrypoint-initdb.d/schema.sql

  redis:
    image: redis:7-alpine
    ports:
      - '6379:6379'

volumes:
  pgdata:
```

9.2 GitHub Actions CI/CD

```yaml
name: Deploy
on:
  push:
    branches: [main]

jobs:
  deploy-frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Deploy to Vercel
        run: npx vercel --prod --token=${{ secrets.VERCEL_TOKEN }}

  deploy-backend:
```

```
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Deploy to Render
        run: curl -X POST ${{ secrets.RENDER_DEPLOY_HOOK }}
```

---

10. Build Sequence (What to Build in What Order)

> *This order matters. Each step should be working and tested before moving to the next. Do not skip ahead. An untested foundation will collapse every feature built on top of it.*

Week	What to Build	Success Criteria	Skip If
1	DB schema + auth (email + Google JWT)	Can register, login, receive JWT, hit protected route	—
1	User profile CRUD + skills	Can update profile, add skills, fetch own profile via API	—
2	Project creation + team system	Can create project with required_skills, team auto-created	—
2	Skill matching algorithm	GET /ai/match/:projectId returns scored user list correctly	—
3	AI Project Advisor	POST /ai/advisor returns valid JSON roadmap for any idea	—
3	Frontend: landing + dashboard + profile	Real users can sign up and see dashboard with data	—
4	Real-time chat (team rooms)	Two users in same project can exchange messages live	—
4	File upload to S3 + resource listing	User can upload PDF, it appears in project resources	—
5	AI Solution Generator + search	Can search users + projects, AI solver returns code	—
5	Anonymous chat + notifications	Anonymous messages post, users receive invite notifications	—
6	Deploy + onboard 20 real users	20 people have accounts with real profiles and skills	—

---

11. Sequential Build Prompts for Antigravity

Use these prompts in order. Do not paste all at once. Each prompt references working output from the previous step. Paste each prompt, get working code, test it locally, then move to the next.

> *If Antigravity generates code that does not run locally, debug it before moving to the next prompt. A broken foundation makes every subsequent prompt harder to fix.*

PROMPT 1 — Database Schema & Backend Scaffold

```
Build the backend foundation for a platform called CONNECT.

TASK: Create a Node.js + Express backend with PostgreSQL.

STACK: Node.js, Express, pg (postgres client), bcryptjs,
jsonwebtoken, passport, passport-google-oauth20, dotenv, cors

CREATE THESE FILES:
1. backend/server.js — Express app with CORS, JSON middleware,
   route mounting, error handler
2. backend/src/config/db.js — pg Pool using DATABASE_URL env var
3. database/schema.sql — Full PostgreSQL schema with these tables:
   users, profiles, skills, user_skills, projects, teams,
   team_members, tasks, chat_rooms, messages, resources, notifications

SCHEMA RULES:
- Use UUID primary keys with gen_random_uuid()
- users.role: CHECK IN ('student','teacher','professional','admin')
- users.provider: CHECK IN ('local','google')
- messages must have room_id FK to chat_rooms (NOT just sender_id)
- chat_rooms has project_id FK and type CHECK IN ('team','anonymous')
- notifications.payload is JSONB
- Add created_at TIMESTAMPTZ DEFAULT NOW() to all tables

ALSO CREATE:
- backend/package.json with all dependencies
- .env.example with all required environment variables
- docker-compose.yml for local PostgreSQL + Redis

OUTPUT: Working backend that starts with npm start and connects to DB.
No fake implementations. All files must be real and complete.
```

PROMPT 2 — Authentication System

```
Using the Express backend and schema from Prompt 1, build the
complete authentication system for CONNECT.

IMPLEMENT:
```

```
1. Email/Password Auth
   POST /api/auth/register
     - Validate email + password (min 8 chars)
     - Hash password with bcrypt (10 rounds)
     - Insert into users table, create empty profile
     - Return { access_token, user: { id, email, role } }

   POST /api/auth/login
     - Find user by email, bcrypt.compare password
     - Return { access_token, user }
     - Set refresh_token as httpOnly cookie

2. Google OAuth
   GET /api/auth/google → redirect to Google
   GET /api/auth/google/callback → create or find user, issue JWT

3. JWT Middleware
   middleware/auth.middleware.js
   - Verify Bearer token from Authorization header
   - Attach decoded user to req.user
   - Return 401 if invalid or expired

4. RBAC Middleware
   middleware/rbac.middleware.js
   - requireRole(['student','professional']) helper

5. POST /api/auth/refresh — use refresh token cookie to issue new access token

ACCESS TOKEN: 15min expiry
REFRESH TOKEN: 7 days, stored in users table as refresh_token column

TEST: Provide curl commands to test register, login, and a protected route.
```

PROMPT 3 — User Profiles & Skill System

```
Using auth middleware from Prompt 2, build the user profile and
skill system for CONNECT.

ENDPOINTS TO BUILD:

GET  /api/users/:id          — public profile (no auth required)
PUT  /api/users/profile      — update own profile (auth required)
  Body: { name, institution, department, year, bio, github_url, linkedin_url }

GET  /api/users/me           — get own full profile (auth required)

POST /api/users/skills       — add skill to own profile
  Body: { skill_name, level } level: beginner|intermediate|advanced
  If skill doesn't exist in skills table, create it first
```

```
DELETE /api/users/skills/:skillId — remove skill from profile

GET /api/users/search?q=term — search users by name or skill name
  Use PostgreSQL ILIKE for name search
  Also search skills table and return users who have matching skills
  Return: [{ id, name, institution, skills[], avatar_url }]

MODEL LAYER: Create backend/src/models/user.model.js with SQL query
functions. Do not use an ORM. Write raw pg queries.

VALIDATION: Sanitize all inputs. Year must be 1-6. Skill level must
be one of the three allowed values.

ALSO: Create a seed file database/seed.sql with 5 sample users,
3 skills, and user_skills entries for testing the match algorithm.
```

PROMPT 4 — Project System & Skill Matching

```
Build the project system and AI skill matching algorithm for CONNECT.
Use the users, skills, and auth from previous prompts.

PROJECT ENDPOINTS:

POST /api/projects           (auth)
  Body: { title, description, required_skills: string[] }
  Creates project + team in one transaction
  Adds creator as first team_member with role='owner'
  Creates a chat_room for the project automatically

GET  /api/projects           (public) — paginated list, ?status=open
GET  /api/projects/:id       (public) — full detail with team members
PUT  /api/projects/:id       (auth, owner only)
DELETE /api/projects/:id     (auth, owner only)

POST /api/projects/:id/join  (auth) — creates a join request notification
POST /api/projects/:id/invite (auth, owner) — invites user by user_id

TASK ENDPOINTS:
POST /api/projects/:id/tasks — create task (auth, team member only)
  Body: { title, description, assigned_to, deadline }
PUT  /api/projects/:id/tasks/:tid — update status (auth, team member)

SKILL MATCHING ENDPOINT:
GET /api/ai/match/:projectId
  - Fetch project.required_skills
  - Fetch all users with their skills
  - For each user compute match_score:
    skill_overlap = count(user skills in required_skills)
    coverage = skill_overlap / required_skills.length
```

```
    level_weights = { beginner: 0.5, intermediate: 1.0, advanced: 1.5 }
    level_score = sum(weights of matched skills) / (matched_count * 1.5)
    match_score = (coverage * 0.6) + (level_score * 0.4)
  - Return top 10 users sorted by match_score desc
  - Each result: { user_id, name, avatar_url, match_score,
    matched_skills[], missing_skills[] }
  - Exclude users already in the team

WRITE UNIT TESTS for the matching algorithm using Jest.
Test at minimum: perfect match, partial match, no match, empty skills.
```

PROMPT 5 — AI Advisor & Solver

```
Build the AI service layer for CONNECT. This connects to OpenAI API.

ENDPOINT 1: POST /api/ai/advisor
  Auth: required
  Rate limit: 5 requests per user per hour (use Redis counter)
  Body: { idea: string }

  SYSTEM PROMPT (use this exactly):
  'You are a senior engineering mentor. Given a project idea, return a
  structured JSON response with these fields:
  - summary: string (one sentence description)
  - recommended_stack: string[] (tech names only)
  - phases: [{ phase_name, tasks: string[], duration_weeks: number }]
  - required_skills: string[] (for team formation)
  - risks: string[]
  Return ONLY valid JSON. No markdown. No explanation outside JSON.'

  MODEL: gpt-4
  Parse JSON response. If parse fails, return 500 with error.
  Never return raw LLM output to client.

ENDPOINT 2: POST /api/ai/solve
  Auth: required
  Rate limit: 10 requests per user per hour
  Body: { problem: string, language?: string }

  SYSTEM PROMPT:
  'You are a senior software engineer. Given a problem, return JSON:
  { explanation: string, code: string, language: string,
    alternatives: [{ name, description }] }
  Return ONLY valid JSON. No markdown.'

  MODEL: gpt-3.5-turbo (cost control)

RATE LIMITER IMPLEMENTATION:
  Use Redis INCR + EXPIRE per user per endpoint.
  Key pattern: ratelimit:{userId}:{endpoint}
  Return 429 with retry_after seconds if limit exceeded.
```

```
ERROR HANDLING: Catch OpenAI errors, return user-friendly messages.
Log all AI errors with user_id and timestamp.
```

PROMPT 6 — Real-Time Chat

```
Build the real-time chat system for CONNECT using Socket.io and Redis.

DEPENDENCIES: socket.io, @socket.io/redis-adapter, ioredis

SETUP IN server.js:
  - Attach Socket.io to HTTP server
  - Configure Redis adapter for pub/sub
  - Import and initialize chat handler

AUTHENTICATION FOR WEBSOCKETS:
  - On connection, verify JWT from socket.handshake.auth.token
  - If invalid, disconnect the socket immediately
  - Attach user data to socket object

SOCKET EVENTS:

  CLIENT emits 'join_room' with { roomId }
  SERVER: verify user is a team_member of the project that owns this room.
          If not, emit 'error' and return.
          Join socket to room. Load last 50 messages from DB.
          Emit 'room_joined' with { roomId, messages: [] }

  CLIENT emits 'send_message' with { roomId, content, isAnon }
  SERVER: validate content not empty, max 2000 chars.
          Insert message into DB (sender_id = null if isAnon).
          Emit 'receive_message' to all in room with:
          { id, sender: { id, name, avatar } or null if anon,
            content, isAnon, createdAt }

  CLIENT emits 'typing' with { roomId }
  SERVER: emit 'user_typing' to others in room with { userName }
          Do NOT save to DB.

HTTP ENDPOINT (for loading history before socket connect):
  GET /api/chat/rooms/:roomId/messages?before=timestamp
  Returns last 50 messages before timestamp (for pagination)

ANONYMOUS CHAT:
  POST /api/chat/anonymous — no auth required
  Body: { content } — max 500 chars
  Inserts into global anonymous room (create one on server start if not exists)
  Rate limit: 1 message per IP per minute (use Redis)
```

PROMPT 7 — Frontend (Next.js)

```
Build the Next.js frontend for CONNECT.

STACK: Next.js 14 (App Router), Tailwind CSS, axios, socket.io-client

THEME:
  Background: #0A0A14
  Primary accent: #00FFC6
  Secondary accent: #00A3FF
  Text: #E0E0E0
  Card background: #12121F
  Border: #1E1E33

PAGES TO BUILD:

1. / (Landing Page)
   - Hero: 'Find Your Team. Build Your Future.'
   - Feature cards: Team Matching, AI Advisor, Real-time Collaboration
   - CTA buttons: Get Started (→ /register), Login (→ /login)
   - No auth required

2. /dashboard (auth required)
   - Sidebar: nav links to all sections
   - My Projects list (cards with status badge)
   - Suggested teammates (top 3 matches for my latest project)
   - Recent notifications

3. /profile/[id]
   - Avatar, name, institution, bio
   - Skill chips with level badges
   - Projects list
   - Edit button if own profile

4. /projects
   - Grid of project cards (title, description snippet, required skills,
     team size, status)
   - Filter by status
   - Create Project button → modal form

5. /projects/[id]
   - Project detail, team members, task board (Kanban: todo/in_progress/done)
   - Matched teammates panel (from /api/ai/match/:id)
   - Tab: Resources, Chat

6. /ai
   - AI Project Advisor: textarea + submit → render roadmap as cards
   - AI Solution Generator: textarea + language select + submit → code block

7. /chat/[roomId]
   - Message history, input bar, send button
   - Show typing indicator
   - Anonymous mode toggle
```

```
GLOBAL:
  - AuthContext with useAuth hook (login, logout, user state)
  - Axios instance with base URL + Authorization header interceptor
  - Protected route wrapper that redirects to /login if no token
  - Responsive: works on mobile (hamburger menu for sidebar)
```

PROMPT 8 — File Upload & Search

```
Build file upload and search for CONNECT.

FILE UPLOAD:
DEPENDENCIES: multer, @aws-sdk/client-s3, @aws-sdk/lib-storage

POST /api/resources
  Auth required
  Multipart form: file + { title, type, project_id? }
  Allowed types: pdf, docx, txt, jpg, png (validate MIME type)
  Max size: 10MB
  Upload to S3: key = uploads/{userId}/{timestamp}-{filename}
  Insert into resources table with file_url = S3 public URL
  Return resource object

GET /api/resources?projectId=&type=
  List resources. If projectId given, filter by project.
  Auth required for project resources.

SEARCH:
GET /api/search?q=term&type=users|projects|all

  Users search:
  SELECT u.id, p.name, p.institution, array_agg(s.name) as skills
  FROM users u JOIN profiles p ON u.id = p.user_id
  LEFT JOIN user_skills us ON u.id = us.user_id
  LEFT JOIN skills s ON us.skill_id = s.id
  WHERE p.name ILIKE '%term%' OR s.name ILIKE '%term%'
  GROUP BY u.id, p.name, p.institution
  LIMIT 20

  Projects search:
  SELECT * FROM projects
  WHERE to_tsvector('english', title || ' ' || COALESCE(description,''))
  @@ plainto_tsquery('english', $1)
  LIMIT 20

  Return unified result: { users: [], projects: [] }

Add search bar to frontend navbar. Show results in dropdown while typing
(debounce 300ms). Clicking result navigates to profile or project page.
```

PROMPT 9 — Security, Polish & Deploy

```
Harden and deploy CONNECT.

SECURITY ADDITIONS:
1. helmet — HTTP security headers
2. express-rate-limit — 100 req/15min per IP globally
3. express-validator — add to all POST/PUT endpoints:
   - Trim all strings
   - Email must be valid email
   - Password min 8 chars
   - SQL injection: pg parameterized queries already handle this,
     but add input validation middleware before all routes
4. CORS: restrict to FRONTEND_URL env var only in production
5. Disable X-Powered-By header
6. Add /api/health endpoint returning { status: 'ok', timestamp }

DEPLOYMENT:

Frontend (Vercel):
  - vercel.json: { 'rewrites': [{ 'source': '/(.*)', 'destination': '/' }] }
  - Set all NEXT_PUBLIC_ env vars in Vercel dashboard

Backend (Render):
  - Create Dockerfile.backend:
    FROM node:20-alpine
    WORKDIR /app
    COPY package*.json ./
    RUN npm ci --only=production
    COPY src/ ./src/
    COPY server.js ./
    EXPOSE 5000
    CMD ['node', 'server.js']

Database (Neon or Supabase):
  - Create project, get connection string
  - Run schema.sql then seed.sql in their SQL editor

GitHub Actions (.github/workflows/deploy.yml):
  - On push to main:
    - Run Jest tests (npm test in backend/)
    - If tests pass, trigger Render deploy webhook
    - Vercel auto-deploys from GitHub

FINAL CHECKLIST before going live:
  [ ] All env vars set in production
  [ ] DATABASE_URL points to cloud DB
  [ ] REDIS_URL points to Upstash
  [ ] CORS FRONTEND_URL is production Vercel URL
  [ ] JWT_SECRET is long random string (not 'secret')
  [ ] S3 bucket has correct CORS policy for uploads
  [ ] Test register + login on production URL
  [ ] Test creating a project and matching teammates
  [ ] Test AI advisor with a real idea
```

12. Risk Register (With Mitigations)

Previous documents listed risks with no response plan. This table pairs each risk with a concrete mitigation strategy.

Risk	Why It Matters	Mitigation
Cold start — no users	Matching is useless with <20 users	Launch in one department only. Get 20 classmates on Day 1 before building more features.
AI hallucination	Bad roadmap destroys trust instantly	Show 'AI-generated, verify before use' label. Let users rate/flag AI output. Log all responses.
OpenAI cost overrun	Unbounded usage = large bill	Hard rate limits per user (10/hr). Use gpt-3.5 for solver. Add monthly spend alert in OpenAI dashboard.
Anonymous chat abuse	Harassment with no accountability	Rate limit: 1 msg/min per IP. Add flag button. Review flagged msgs manually. Ban IPs if needed.
Data sparsity on skills	Few skills = bad matching = users leave	Pre-seed skills table with top 50 engineering skills. Suggest skills on onboarding.
No traction after launch	Building more features won't fix distribution	Interview 10 users in week 1. If matching unused, talk to them. Pivot feature priority based on real behavior.

13. Scaling Roadmap (After Traction)

> *Do not build any of this until you have 200+ active users and a specific performance bottleneck. Building this early is wasted engineering time.*

Stage	Trigger	What to Add
Stage 2	DB queries > 500ms p95	Add pgvector for semantic search. Add read replica. Query optimization.
Stage 2	WebSocket connections > 1000	Move to Redis cluster. Add Socket.io sticky sessions.
Stage 3	AI service causing backend timeouts	Extract AI service to separate deployment. Add job queue (Bull + Redis).

Stage 3	File uploads slowing main server	Move uploads to pre-signed S3 URLs (client uploads directly to S3).
Stage 4	> 10,000 users	Kubernetes. Horizontal pod autoscaling. CDN for static assets.

---

14. Definition of Done (MVP)

CONNECT MVP is complete when ALL of these are true:

1. A user can register with email or Google, fill their profile, and add skills
2. A user can create a project with required skills and see matched teammates ranked by score
3. Two users in the same project can chat in real time
4. A user can paste a project idea and receive a structured AI-generated roadmap
5. A user can upload a file to a project and team members can see it
6. All of the above works on the production URL, not just localhost
7. 20 real people (not the builder) have used it and at least one has formed a team

Until all 7 conditions are met, it is not done. It is in progress.

Build the smallest thing that solves the real problem.

Then expand from traction, not from ambition.